

Welcome to Computer Systems, Data Structures and Data Management (4CM508)

Ioannis Tsioulis (MC)



slides acknowledgements:
Dr Sam O'Neill

Instructor Contact & Resources

Ioannis Tsioulis



i.tsioulis@mc-class.gr



github.com/giannis00

Disclaimer

- We will use some code in this lecture, but you don't need to understand it.
- Just the idea of what it is doing!

The Birthday Problem

What is the probability that in a room of n people, at least two people will share a birthday?

Assumptions

- Birthdays are equally distributed
- No leap years, sorry if you have a birthday on Feb 28th!

Guarantee

- Is there a number of people in a room that guarantees a match?

Guarantee

- Is there a number of people in a room that guarantees a match?
- 366 - Pigeon hole principle
 - Imagine 365 boxes, keep putting people in the boxes.
 - The best case is 365 have different birthdays.
 - Hence the 366 is a duplicate.

What about this room?

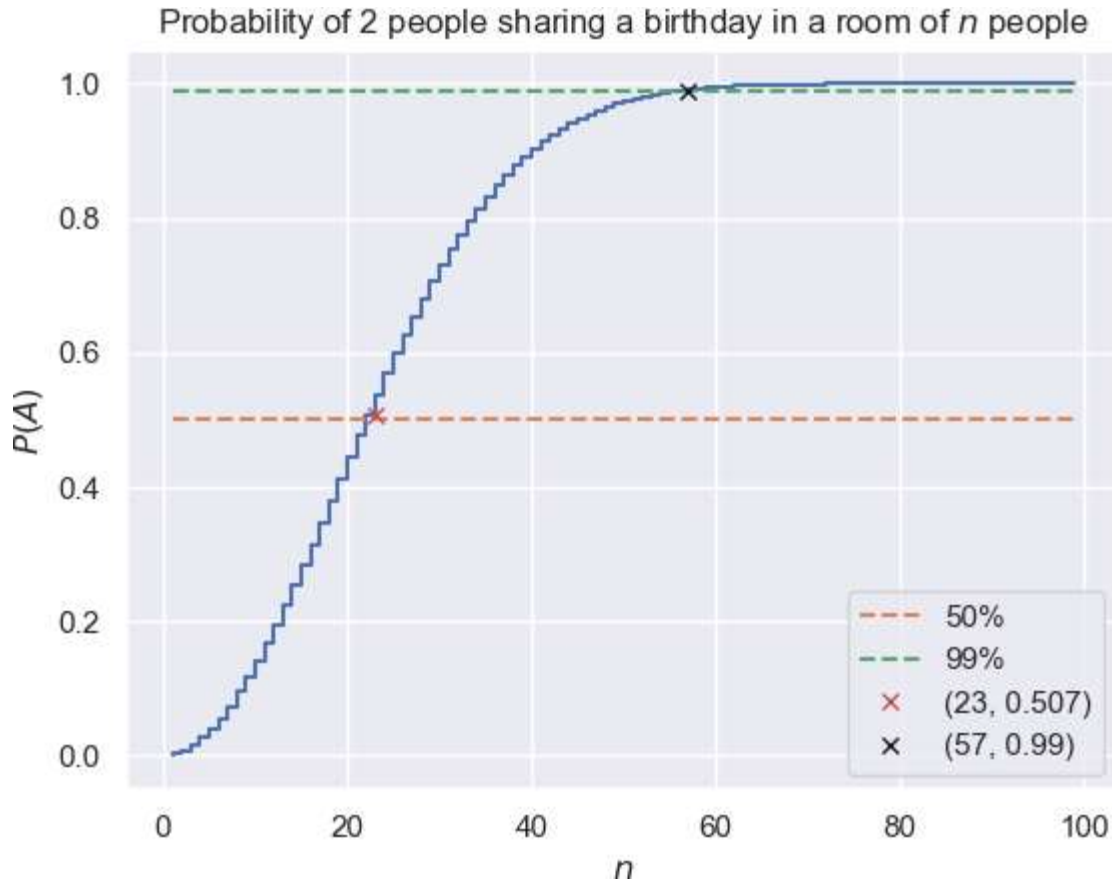
Yes or No?

Let's check...

A Crazy Result

n people in room	Probability of at least one match
1	0%
5	2.7%
10	11.7%
20	41.1%
23	50.7%
30	70.6%
40	89.1%
50	97%

Plotting probabilities for $n = 1, 2, 3, \dots, 365$



Remarkably at **23** people in a room the chance of two people sharing a birthday is over **50%**. At **57** people it is over **99%**. Mad!

Do you trust me?

If not, how can we confirm the previous plot?

Experiment

Here is the idea.

Repeat the following experiment for a value of n (people in room) many times and compute the average result.

- Find a room with n people in
- Check if there is a duplicate
- Record the result as `True` or `False`

For example we for $n = 10$ we might do for 5 experiments:

`True, False, False, True, False`

Therefore we have an approximation of the probability to be $2/5$ or 40%.

How can we check for a duplicate (match)?

- Ideas?

Idea 1

1. Ask the first person what their birthday is?
2. Check this person against all other people for a match
3. Repeat this with the next person in the room

Is this any good? Is this efficient?

This was a bad idea!

For a room of 5 people, idea 1 has to check the following.

Person	Checks Against Persons
1	2, 3, 4, 5
2	3, 4, 5
3	4, 5
4	5

It is checking all possible pairs of people!

For a room with n people in it, this is approximately $\frac{n^2}{2}$

A better way - Idea 2

Create 365 boxes for the days of the year

1. Ask first person their birthday and write it down in a box of 365 boxes
2. For every other student
 - Ask next person their birthday
 - if birthday box has a birthday in we have a match - end!
 - else put birthday in new box

This is clearly better. Worst case we check every person in the room

Law of Large Numbers

The more repetitions (trials) we do the closer we will get to the true probability.

- But how do we find thousands of different rooms of n people?
- Won't that take a lot of time!?

Experiment Python Code

- We can use code to set up a simulation of the experiment.
- The code on the next slide implements **idea 1**.

Experiment Python Code

```
import random

def generate_rand_list(n):
    return [random.randint(1,365) for _ in range(n)]

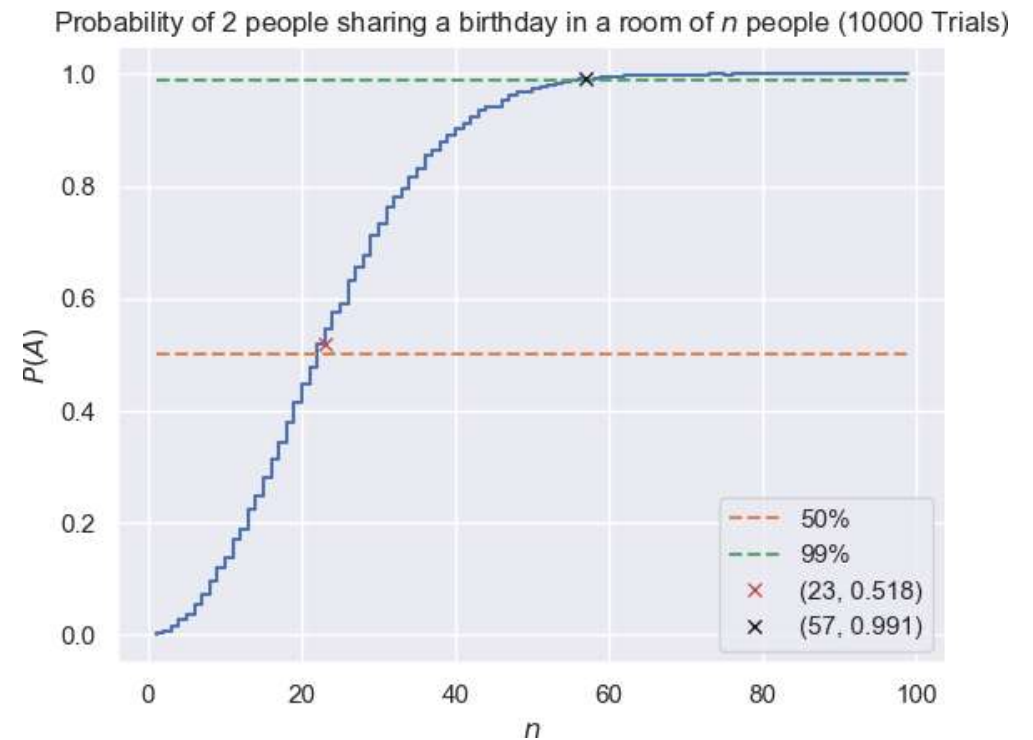
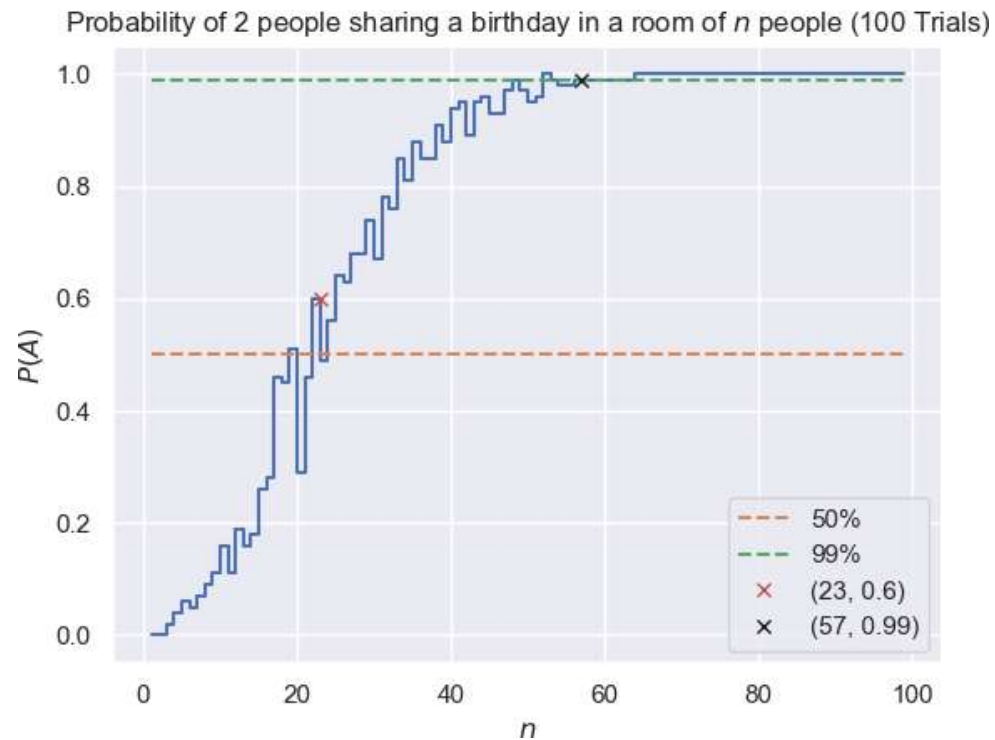
def has_duplicate1(l): # idea 1
    """ Take a list l and check if it contains a duplicate. """
    has_duplicate = False
    for i in range(len(l)):
        for j in range(i+1, len(l)):
            if l[i] == l[j]:
                has_duplicate = True
    return has_duplicate

def run_trials(n, k):
    no_successes = 0
    for _ in range(k):
        rand_list = generate_rand_list(n)
        if has_duplicate1(rand_list):
            no_successes += 1
    return no_successes/k

k=100
for n in range(1,5):
    print(round(run_trials(n,k),3))
```

Let's try this

Experimental Results - Look familiar?



Key Questions

1. Is this the best way to do this?
2. How do we measure best?
3. How else might we implement it?

A better way

Here is the Python code to implement **idea 2**.

```
def has_duplicate2(birthday_list): # idea 2
    """ Take a list l and check if it contains a duplicate using set conversion """
    # create 365 boxes
    boxes = [0]*365

    # for each birthday in the list l
    for birthday in birthday_list:
        # put birthday in correct box
        boxes[birthday-1] += 1

        # check if box contains more than 1
        if boxes[birthday-1] > 1:
            return True

    return False
```

Which is faster?

We think based on our analysis that **idea 2** is better than **idea 1**

Let's test this!

Which is faster?

`has_duplicate_1()` takes more than 3 seconds to complete for a list of length 10000.

`has_duplicate_2()` takes a fraction of a second, in fact about 0.00001 seconds.

Why is it faster?

For a list of length 5, `has_duplicate_1()` has to check the following.

Item	Checks Against Items
1	2, 3, 4, 5
2	3, 4, 5
3	4, 5
4	5

It is checking all possible pairs of people!

For a list of length n it is approximately $\frac{n^2}{2}$

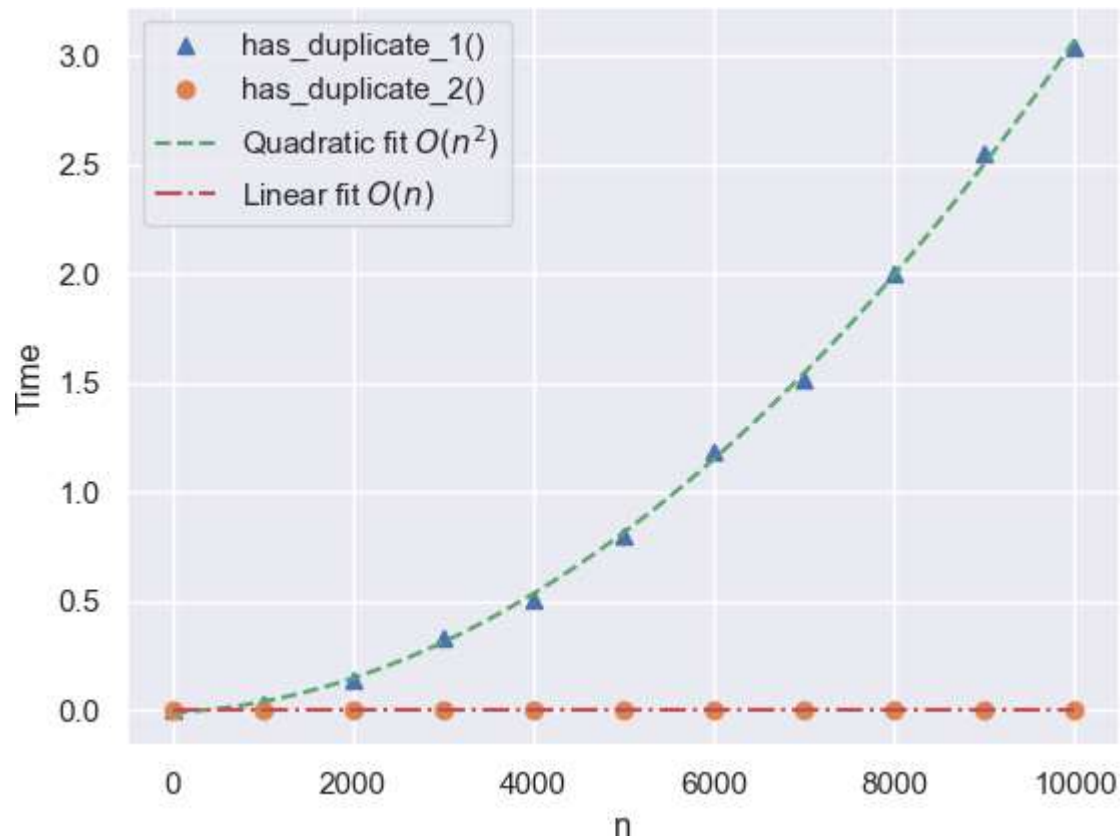
Why is it faster?

For a list of length 5, `has_duplicate_2()` has to (at most) check each item once. That is (at most) 5 checks

For a list of length n it is approximately n

Plotting different values of n

If you plot the performance for different lengths of lists n (number of people in a room) you see `has_duplicate_1()` is much worst as n gets large.



The importance of this cannot be understated in writing efficient programs. Is your idea any good?

It is also common to get interview questions on the topics studied in this module, so that is another reason to study it.

NOTE: The programs in this lecture aren't efficient, that was point!

Summary

- We can use an **algorithm** to operate on the data structure.
- **Algorithms** take different times and we can **measure** them in terms of their input size n .
- Irrespective of which computer you are on, you know one algorithm is better than the other and roughly by how much. **Count the operations!**
- You can work this out without even timing the algorithms. **More on this next lecture.**

Please read the reading for this lecture

Additional Stuff

I have provided some insight into the probability if you are interested on the remaining slides.

You can just ignore this if you want!

Probability

Let A be the event that two people share a birthday in a room of size n .

$P(A)$ is the probability of event A happening.

This is not a probability class, you can look this and the following up. I won't be explaining it properly!

Key Insight

The key to this problem is to think about the probability of the event not happening.
That is...

In a given room of size n what is the chance that no two people have the same birthday? i.e. everyone has a different birthday

Let's call that event A' .

Then $P(A) = 1 - P(A')$

This is because A and A' can't both happen. Think about it.

Computing the probability $n = 2$

Let's start with a room of 2 people. What is the probability of $P(A)$?

- Let's first calculate $P(A')$.
- The first event is that person 1 has a birthday with probability $\frac{365}{365} = 1$.
- The second event is that person 2 has a different birthday. Well, 1 of the 365 days is taken, so the probability of this is $\frac{364}{365}$.
- By conditional probability we have:
 - $P(A') = \frac{365}{365} \times \frac{364}{365} = \frac{364}{365} = 0.997$ (3.d.p)
- Therefore,
 - $P(A) = 1 - \frac{364}{365} = 0.003$ or 0.3%

Computing the probability $n = 3$

Room of 3 people

$$P(A') = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} = 0.992 \text{ (3.d.p)}$$

Therefore,

$$P(A) = 1 - 0.992 = 0.008 \text{ or } 0.8\%$$

Computing the probability n

In general for a room with n people, we have,

$$P(A) = 1 - \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \cdots \times \frac{365 - (n - 1)}{365}$$

Computing different values of n with Python

```
def prob_same_birthday(n):  
    prob_result = 1  
    for x in range(364, 365-n, -1):  
        prob_result *= x/365  
  
    return 1 - prob_result  
  
for n in range(1,5):  
    print(round(prob_same_birthday(n),3))
```

```
0  
0.003  
0.008  
0.016
```